PREFER: Persistent-Frequency KV-Cache Eviction for LLM Prefix Caching

1st Watin Promfiy
School of Information Technology
King Mongkut's Institute of Technology Ladkrabang
Bangkok, Thailand
66070184@kmitl.ac.th

2nd Chaloemphon Sirikayon
School of Information Technology
King Mongkut's Institute of Technology Ladkrabang
Bangkok, Thailand
chaloemphon@it.kmitl.ac.th
*Corresponding author

3rd Panwit Tuwanut
School of Information Technology
King Mongkut's Institute of Technology Ladkrabang
Bangkok, Thailand
panwit@it.kmitl.ac.th

4th Suvit Poomrittigul
School of Information Technology
King Mongkut's Institute of Technology Ladkrabang
Bangkok, Thailand
suvit@it.kmitl.ac.th

***Abstract*—Prefix caching of attention key/value (KV) state is standard in large language model serving, yet vLLM and SGLang both default to Least Recently Used (LRU) eviction. Holding the prefix-cache model fixed, we ask how far LRU is from a strong achievable reference and whether a searched eviction policy closes the gap. We build a radix block-cache simulator validated against a live engine, a prefix-constrained Belady reference (feasible, not a proven optimum), and an infinite-cache ceiling. On two production traces an automated search converges on frequency-dominated eviction (PREFER); the searched policy closes 14–27% of the LRU-to-reference gap on held-out data at 1k–4k-block capacities and reaches 1.9× LRU's hit rate on conversation at 2k blocks. Comparison with LFU, ARC, and LIRS isolates the operative ingredient: access frequency that *persists across evictions*; non-persistent LFU and ARC stay within 7% of LRU on traces whose reuse distances reach thousands of requests. Live, over 8 seeds on a consumer GPU, frequency eviction raises the hit rate by 25–31% on constructed multi-tenant and session-resumption workloads; at their short reuse distances SGLang's built-in, non-persistent LFU (off by default) already gains 25–29%, and our persistent-frequency patch adds no significant further gain there. Both lose on a recency-heavy agent loop. An advisor that simulates a workload sample selects the winning policy in all four live workload families. The gain carries to a second engine and to a retrieval-augmented vision-language application, where it yields up to 10.5% more throughput on an 8B model under load.**

***Index Terms*—large language models, inference serving, KV cache, prefix caching, cache eviction, Belady**

I. Introduction

Serving large language models (LLMs) is increasingly dominated by *prefill*: shared system prompts, few-shot exemplars, multi-turn histories, and agent scratchpads mean that a large fraction of prompt tokens recur across requests. Production engines exploit this with prefix caching, storing key/value (KV) blocks in a prefix or radix structure and reusing the longest cached prefix of each request. When the KV pool is full, blocks must be evicted, and the two most widely deployed open systems both evict by *Least Recently Used* (LRU) by default.

LRU is the reflexive default, but caching theory has long known it can be far from optimal when reference popularity is not recency-correlated. We make this concrete for LLM prefix caching (Fig. 1). Our contributions are: (i) a gap-to-reference framework consisting of a validated radix block-cache simulator, a prefix-constrained Belady reference, and an infinite-cache ceiling; (ii) an automated policy search that converges on frequency-dominated eviction (PREFER, Prefix REuse-Frequency Eviction and Retention), compared against LFU, ARC, and LIRS adapted to the prefix constraint, which identifies *persistent* frequency as the operative ingredient; (iii) a live study over 8 seeds, with tail-latency and throughput intervals, of LRU, the engine's built-in per-node LFU, and a small patch that makes counts persist across eviction, showing that at short reuse distances the existing option already suffices; (iv) a workload-dependence analysis and an *advisor* that selects the policy by simulating a workload sample, validated against live outcomes; and (v) cross-engine and cross-application validation plus two honest negatives (a learned reuse predictor and a non-linear policy).

We are explicit about what we do *not* claim: this is not a throughput win over production systems as a whole. It is an apples-to-apples comparison of the eviction scoring function with the cache model held fixed. Section II reviews related work and the traces, Sections III and IV define the references and the search, Section V describes the setup, and Sections VI–

979-8-3195-1044-0/26/$31.00 ©2026 IEEE

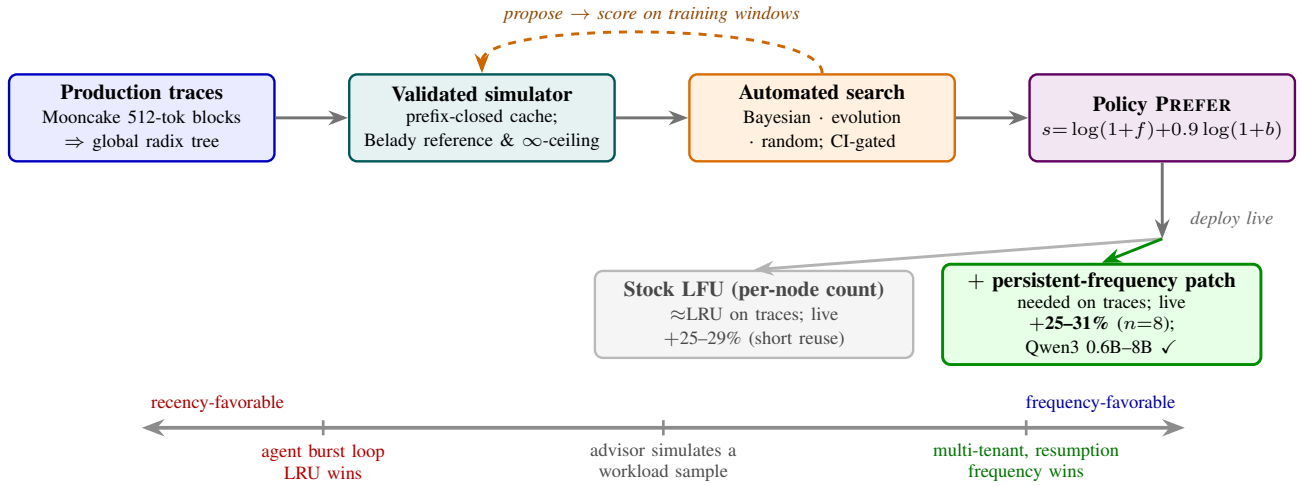


Fig. 1. **System overview.** *(Top)* Public Mooncake traces induce a global radix tree; a simulator validated against a live engine scores eviction policies against a Belady reference and an infinite-cache ceiling; an automated, CI-gated search on training windows converges on frequency-dominated eviction. *(Middle)* The engine's built-in LFU keeps a count only while a node is resident: it barely beats LRU on the long-reuse traces but matches PREFER on the short-reuse live workloads; persistent counts are needed for the former (live: mean hit-rate gain over LRU). *(Bottom)* The winner is workload-dependent; an advisor chooses by simulation.

VIII report the results, negative results, and limitations.

II. Background and Related Work

Prefix caching in LLM serving. vLLM's Automatic Prefix Caching [1] hashes fixed-size KV blocks together with their prefix and reuses resident blocks, evicting LRU. SGLang's RadixAttention [2] stores KV in a radix tree, evicts the least-recently-used leaf, and adds cache-aware scheduling. Prompt Cache [3] reuses attention state for repeated prompt modules, and CachedAttention [4] extends reuse across multi-turn conversations by staging KV in a slower memory tier. At a fixed block size their GPU-tier reclamation reduces to LRU eviction over a prefix-closed cache, which is the object of our study. We vary only the scoring function, so our contribution is orthogonal to, and composable with, their scheduling and tiering.

Classical eviction and our novelty. LRU, LFU, GDSF [5], ARC [6], and LIRS [7] span the recency–frequency spectrum; ARC and LIRS balance the two adaptively, but their frequency signal comes from a history bounded to the order of the cache size (ARC's ghost lists, LIRS's recency stack). The distinction between counting references only while an object is resident (in-cache LFU) and counting all references (perfect LFU) is classical in web caching [8], and TinyLFU [9] keeps approximate, aged frequency outside the cache for admission. Our contribution is not a new counter. It is to show that production LLM prefix traces sit in the regime where persistent counts matter and that shipping engines implement the in-cache variant (which suffices only when reuses fall within the cache's residence time), to adapt the policies to the prefix-closed constraint, to measure the gap against a reference, and to select the policy per workload. Belady's MIN [10] is the offline optimum for uniform-cost demand paging, which modern hardware replacement imitates (Hawkeye [11]); learned admission and eviction such as LRB [12] can beat heuristics when reuse is predictable. Two features distinguish our setting from flat paging: references are *prefix-closed* (a block is useful only with its whole ancestor chain resident), and block cost is non-uniform.

Production traces. We use the public, anonymized Mooncake KV-block traces [13], sampled from production traffic of Moonshot AI's Kimi assistant: one conversation and one tool/agent trace of about an hour each. Each request carries an arrival time, prompt and output lengths, and a list of 512-token block identifiers; we replay requests in arrival order. Shared identifiers imply identical prefixes, so they induce a global radix tree. The traces carry no tenant, session, or application labels.

III. Problem and Reference Bounds

Cache model. Each request is a root-to-leaf path of 512-token blocks; the cached set is kept *prefix-closed*, a request's hit is its longest cached prefix, eviction removes a leaf, and capacity is counted in blocks. We report the token-weighted hit rate (cached prefill tokens over total prefill tokens). Both traces form a single consistent tree (zero parent inconsistencies).

References. The true offline optimum under leaf eviction is combinatorial and we do not compute it. The *infinite-cache ceiling* (reuse rate with unlimited cache) is a valid upper bound on any finite-cache hit rate. The *Belady reference* applies Belady's rule inside the prefix model, evicting the evictable leaf whose next reference is farthest. It is a *feasible* schedule under the same capacity, hence a lower bound on the optimum and a strong achievable target, *not* the optimum itself. Consequently "fraction of reference" and "gap closed" overstate the corresponding quantities relative to the unknown optimum. (A branch-and-bound construct and a flat demand-paging bound

were explored but are unsound on small pathological instances and are not used.)

IV. Automated Policy Search

Policy family. Eviction removes the evictable leaf with the lowest score, a weighted sum of five causal, order-one-scaled block features: recency (last-access index over horizon), frequency (log access count, persistent across evictions), depth (one over one plus depth), branchiness (log distinct children seen), and a system-prompt flag. LRU is recency alone; LFU is frequency alone. Each feature changes only at a block's own accesses, so eviction uses an exact lazy min-heap: $O(\log n)$ amortized per access for n resident blocks, the same order as LRU in the simulator. Fig. 2 works one eviction step.

Search loop and data hygiene. (i) Hand-designed seeds (LRU, LFU, depth, and blends); (ii) random exploration; (iii) directed search by Bayesian optimization (Gaussian process, expected improvement), compared against an evolution strategy and random search. The objective is the mean fraction-of-reference over five training sub-windows and three capacities (512, 2k, and 16k blocks for the final policy; 1k and 4k in the optimizer comparison). A candidate becomes the incumbent only if it beats it under a one-sided paired test across the evaluation points; search stops on a patience budget over the *training* objective. Every twenty iterations the incumbent is also scored on the held-out split, but this score is only *logged* for monitoring: it never enters acceptance, the optimizer, or stopping. The held-out split is the last 20% of each trace in time. All randomness is seeded.

V. Experimental Setup

Traces. Table I characterizes the workloads. Each Mooncake trace spans 59 min (mean inter-arrival 294 ms conversation, 150 ms tool/agent; 24 and 17 blocks per request); 76% and 78% of distinct blocks are referenced once. We sweep capacities of 256–16,384 blocks.

Constructed live workloads. Multi-tenant serving, agent session-resumption, and the burst loop are *constructed* request generators, not phenomena observed in the traces (which carry no tenant or session labels). Multi-tenant: 400 requests, each a Zipf(a=1.1)-chosen template out of 12 followed by a unique 45-passage context. Resumption: 60 Zipf-weighted tool-calling sessions of 8 steps each with a 30-request cool-down between steps of one session. Burst loop: the same agents (24 sessions of 6 steps) with no cool-down. Replies are generated by the model and appended to the session.

Engines and evidence classes. All live runs use one NVIDIA RTX 5070 Ti (16 GB). To separate what supports each claim we tag results: [S] simulator on Mooncake traces; [E0] SGLang without our patch (LRU, its built-in LFU with per-node `hit_count`, or an in-cache scoring strategy); [E1] SGLang with the persistent-frequency patch and PREFER scoring from our `preferkv` package; [V] vLLM with the same patch; [P] application replay with a text-only proxy model. Table IV ([E0] vs. [E1]) uses SGLang v0.5.18 (CUDA 12.9), Qwen2.5-1.5B-Instruct in BF16, Triton attention with CUDA graphs disabled, KV budget set in tokens (`--max-total-tokens`; radix page size 1 token), concurrency 1, greedy decoding with at most 32/48/64 output tokens, and seeds 0–7, each seed fixing the request schedule replayed to all three policies. Stock LFU is `--radix-eviction-policy lfu`; PREFER uses the same flag with the patched cache and our scoring. Each policy run starts a fresh server with one warm-up request, and the policy order is rotated across seeds. Requests are issued closed-loop (each sent when the previous completes). TTFT is measured client-side from request send to the first streamed token, and p99 TTFT is the nearest-rank percentile over a run's 144–480 requests; throughput is requests over the run's wall-clock time; hit rate is the engine-reported cached prompt tokens over prompt tokens. Relative changes are computed per seed against that seed's LRU run, then averaged. Earlier live results (Sec. VI-C, VI-E) used a June-2026 SGLang development build, vLLM v0.24.0 (16-token blocks), and Qwen2.5 0.5–3B and Qwen3 0.6–8B (8B in FP8), at concurrency 1 unless stated; their latency is whole-request latency with short outputs.

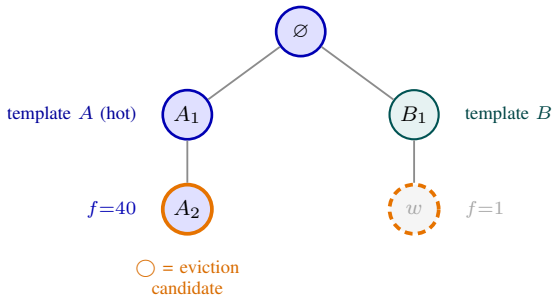
TABLE I
Workload characterization. (M) Mooncake trace, 512-token blocks; (C) constructed live workload, statistics over shared prefixes (templates/sessions) for seeds 0–7. One-shot: share of block references to once-referenced blocks (M), or of prompt tokens that are new (C). Top-1%: reference share of the most popular 1% of blocks. Reuse distance in requests; near: reuses within 8 requests.

workload	req.	one-shot %	top-1% %	reuse dist. p50/p90	near %
conversation (M)	12,031	48	12	384 / 1,748	11
tool/agent (M)	23,608	35	40	4 / 1,630	58
multi-tenant (C)	400	29	–	5 / 29	66
resumption (C)	480	13	–	38 / 86	1
burst loop (C)	144	16	–	9 / 35	49

VI. Results

A. Frequency beats LRU; persistence is what matters [S]

Table II compares, on held-out data, LRU with ARC, LIRS, LFU with counts reset on eviction (in-cache), LFU with persistent counts, the deployed PREFER form, and the searched weight vector. At 1k–4k blocks the searched policy closes 14–27% of the LRU-to-reference gap and persistent LFU 13–26%, while ARC (0–3%) and in-cache LFU (0–2%) barely move from LRU. At capacity 2048 the searched policy reaches 0.43 of the reference on conversation (LRU 0.22) and 0.79 on tool/agent (LRU 0.73). LIRS is the strongest classical baseline: it trails PREFER below 4k blocks (.062 vs. .095 on conversation at 2k) but matches it at 4k (.135 vs. .134; .428 vs. .425), and it falls below LRU at 16k (.223 vs. .249; .487 vs. .517), where PREFER stays at or above LRU (.257, .519). (SGLang's radix LRU is within 0.01 of LRU throughout.) The policies that win are those whose frequency memory extends well beyond the resident set: about three quarters of blocks are one-shot, and the reused ones return after hundreds of



Eviction score $s = \log(1+f) + 0.9\,\log(1+b)$ (recency breaks ties)

candidate	role	f	s	last used
A_2	hot prefix, now a leaf	40	**3.71**	long ago
w	one-shot tail	1	0.69	just now

PREFER: evict $\arg\min s = w$ ✓ keeps A_2
LRU: evict oldest $= A_2$ × drops the hot prefix

Why A_2 still scores high: persistent frequency. A side table keeps $\texttt{hash}(A_1A_2) \to 40$, accumulated over A's reuses and *retained even after A's earlier tails were evicted*. On a stock engine this count dies with the node. **Next request for A_1A_2: PREFER hit / LRU miss.**

Fig. 2. **PREFER on one eviction step (worked example).** Both candidates are childless leaves (b=0), so the score reduces to $\log(1+f)$. The cache is full and must drop a leaf: the *hot* prefix A_2, reused often (persistent frequency f=40) but not recently, or a *fresh one-shot* tail w (f=1). PREFER evicts w and keeps A_2; LRU evicts the older A_2 and misses when A is reused next. The persistent-frequency side table is what lets A_2 keep f=40 across earlier evictions.

TABLE II
TOKEN HIT RATE ON HELD-OUT DATA ([S]); CAPACITY IN 512-TOKEN BLOCKS. LFU-IC: IN-CACHE LFU (COUNT RESET ON EVICTION); LFU-P: PERSISTENT COUNTS; PREFER: DEPLOYED FORM; SRCH.: SEARCHED WEIGHTS; REF.: BELADY REFERENCE; CEIL.: INFINITE CACHE. BEST ONLINE POLICY PER ROW IN BOLD.

trace/cap	LRU	ARC	LIRS	LFU-ic	LFU-p	PREFER	srch.	ref.	ceil.
conv. 1k	.045	.045	.046	.045	.061	.059	**.063**	.171	.286
conv. 2k	.052	.055	.062	.055	.099	.095	**.101**	.236	.286
conv. 4k	.079	.085	**.135**	.082	.133	**.134**	.132	.286	.286
tool 1k	.358	.358	.359	.359	.372	.370	**.374**	.451	.531
tool 2k	.365	.367	.376	.367	**.395**	.393	**.395**	.498	.531
tool 4k	.387	.391	**.428**	.387	.425	.425	.426	.531	.531

TABLE III
FEATURE ABLATION AT CAPACITY 2048 ([S], HELD-OUT). LEFT: HIT-RATE DROP WHEN A FEATURE IS REMOVED FROM THE SEARCHED POLICY. RIGHT: HIT RATE WITH THE FEATURE ALONE, WITHOUT TIE-BREAKING (LFU-P IN TABLE II BREAKS TIES BY RECENCY).

	leave-one-out drop		single-feature hit	
feature	conv.	tool.	conv.	tool.
recency	4.8%	0.0%	0.052	0.365
frequency	**10.4%**	**6.2%**	**0.097**	**0.395**
depth	0.9%	0.0%	0.074	0.349
branchiness	1.0%	0.0%	0.094	0.392
system flag	0.0%	0.0%	0.056	0.370

requests on conversation (Table I). Relative to persistent LFU alone, the deployed PREFER form is within −4.4 to +2.6% and the searched weights within −1.3 to +3.9% across 1k–16k blocks. PREFER therefore has no material benefit over persistent LFU: it is persistent LFU adapted to the prefix tree (leaf-only eviction, recency tie-break), and the contribution is identifying persistence, not the secondary terms.

B. What the search learned, and sensitivity [S]

The optimizers agree: frequency dominates, recency is low or negative. On tool/agent the search converges to pure frequency (weights recency, frequency, depth, branchiness, system = 0, 1, 0, 0, 0); on conversation it adds a moderate recency and a large branchiness weight (0.36, 0.96, 0.05, 0.99, 0.03). The deployed form $s = \log(1+f) + 0.9\log(1+b)$ with recency tie-breaking simplifies these weights. Its secondary terms matter little: varying the branchiness weight over $[0, 3]$ changes hit rate by at most 5.4% (relative) at 1k–4k blocks, and a recency weight up to half the frequency weight changes it by at most 5.7%; only a recency-dominated score (recency weight 2× frequency) degrades it, by up to 30%, toward LRU. On the engine, branchiness is in addition constant across candidates, because only childless leaves are evictable there, so live PREFER is persistent-frequency eviction with recency tie-breaking. A leave-one-out ablation (Table III) agrees that frequency is the dominant feature. Across five search seeds on conversation the training objective is 0.6155 (s.d. 0.0025) and the frequency weight 0.81 (s.d. 0.12). In the optimizer comparison (conversation, 1k and 4k blocks), Bayesian optimization (0.488) edges the evolution strategy (0.479) and random search (0.469), all above the best hand seed (0.460).

C. Simulator fidelity on a live engine [E0]

SGLang exposes the eviction policy as a flag, so the simulator can be checked against the engine. *The simulator predicts the real LRU hit rate to within one percent* (predicted 0.17, measured 0.171). In a deep-request regime (Qwen2.5-0.5B, 8 concurrent requests) an early in-cache frequency strategy on the stock engine (branchiness proxied by the parent's fan-out) gives a hit-rate gain of $10.9 \pm 0.5\%$ (mean ± s.d., three runs; LRU 0.071, frequency 0.079) at neutral latency, and a KV-budget sweep reproduces the simulator's inverted-U gap (+1.7%, +8.4%, +7.3% at 256, 512, 768 blocks). With a real 1.5B model and chat text the cache hits exactly on shared content (88.9%), and 0.5–3B models in three languages give identical per-language hit rates.

D. Realizing frequency on the engine [E0] vs. [E1]

Stock SGLang offers LFU eviction (LRU is the default) with a per-node hit count that is *lost when the node is evicted*, i.e., the in-cache LFU of Table II. Our patch adds a global prefix-to-count map, keyed by a rolling hash of the prefix tokens and updated on the insert path, that survives eviction; the eviction strategy reads it. Table IV runs LRU, stock LFU, and

TABLE IV
LIVE ENGINE ([E0]/[E1], SGLANG V0.5.18, QWEN2.5-1.5B, CONCURRENCY 1): LRU HIT RATE, AND RELATIVE CHANGE VS. LRU IN % (MEAN ± 95% CI OVER 8 SEEDS; POSITIVE IS BETTER) OF HIT RATE FOR STOCK LFU AND PREFER, AND OF PREFER'S MEAN TTFT REDUCTION, P99 TTFT REDUCTION, AND THROUGHPUT GAIN. BOLD: THE HIGHER POSITIVE HIT-RATE GAIN.

	LRU	hit-rate gain		PREFER latency / throughput		
workload, KV	hit	stock LFU	PREFER	TTFT	p99 TTFT	tput
multi-tenant 20k	.410	$+29.0\pm3.6$	$\mathbf{+30.5\pm4.7}$	$+16.6\pm7.1$	$+3.7\pm5.1$	$+7.2\pm6.8$
multi-tenant 24k	.454	$\mathbf{+26.0\pm3.6}$	$+25.4\pm4.4$	$+12.8\pm2.3$	$+4.5\pm2.6$	$+2.6\pm1.8$
resumption 6k	.367	$\mathbf{+27.9\pm1.9}$	$+27.3\pm1.9$	-0.3 ± 1.3	$+2.5\pm6.2$	$+1.7\pm1.6$
resumption 8k	.410	$\mathbf{+25.1\pm3.0}$	$+24.9\pm3.1$	$+1.1\pm1.2$	$+2.3\pm4.3$	$+1.5\pm1.5$
burst loop 6k	.854	-15.3 ± 1.8	-15.3 ± 1.8	-0.9 ± 2.9	-3.3 ± 16.1	$+0.7\pm1.3$
burst loop 8k	.877	-7.4 ± 0.7	-7.4 ± 0.7	-1.4 ± 2.6	-0.2 ± 9.1	$+0.5\pm1.7$

PREFER on the same request streams. On the two frequency-favorable workloads PREFER raises the hit rate over LRU by 25–31% (multi-tenant) and 25–27% (resumption). Unlike on the traces, stock LFU captures essentially all of it (26–29% and 25–28%): these workloads reuse prefixes within tens of requests (Table I), so popular prefixes are rarely evicted before reuse and in-cache counts suffice. The added effect of persistence is not significant: over stock LFU it changes the hit rate by $+1.2\pm3.0\%$ and $-0.5\pm3.0\%$ on multi-tenant serving (20k/24k) and lowers it by at most 0.4% on resumption. On the burst loop both lose to LRU (-15.3%/-7.4% at 6k/8k) with identical hit rates: finished sessions never return, so persistent and per-node counts rank candidates alike. Thus on short-reuse workloads enabling the engine's existing LFU recovers the gain, while persistence is what separates frequency from LRU at the trace-level reuse distances of Sec. VI-A. The reported intervals are 95% Student-t intervals over 8 seeds; each seed is a different request stream, so the intervals include workload variation, and latency is measured on one GPU at concurrency 1. On the earlier development build, the original patch with the same package gives an identical $+31.2\%$ multi-tenant hit-rate gain over LRU for Qwen3 0.6B, 1.7B, 4B, and 8B (FP8) at a 16k budget (single runs), with mean latency reductions of 3.9, 5.2, 9.3, and 8.1%, larger for the larger models as prefill gets more expensive.

Overhead and metadata. The patch adds a rolling hash over each inserted prefix (work proportional to the prefix match the engine already performs) and one map update per node on the insert path; the eviction score is $O(1)$ at the node with no tree walk. A compact table needs 16 bytes per entry (64-bit key and count); the Python dictionary in our prototype costs several times more. Unbounded, the map grows with distinct prefixes: about 187k entries (3.0 MB) per hour on the Mooncake traces. Bounding it at a fixed number of entries and halving all counts when full (dropping zeros) keeps memory constant; in simulation a bound of $8\times$ the cache's block count costs at most 2.1% relative hit rate and $1\times$ at most 6.3%.

E. Cross-application, load, and cross-engine [P], [V]

A real application. We replay the retrieval trace of a dermatology VLM pipeline in which retrieved reference cases are prepended to each query, so popular references form reusable prefixes. Under a skewed (Zipf) serving stream at a pressured budget, SGLang serving Qwen2.5-0.5B, with fixed text tokens in place of each image, measures $+14.5\%$ hit rate over LRU (the simulator agrees within half a point); on the application's *one-pass* offline evaluation the gain is only $+2$–5%.

From hit rate to throughput under load. On the same application stream with Qwen3-8B in FP8 at a fixed 24k-token budget, the hit-rate gain is load-invariant ($+9.4$–10.2% from concurrency 1 to 32) while its conversion grows with load: $+3.9\%$ throughput at concurrency 1, $+8.3\%$ at 16, $+10.5\%$ at 32 (single runs per point), then compressing at 64 where running requests pin most of the pool.

Cross-engine. Dropping the same persistent-frequency scoring into vLLM's *flat* block pool yields $+6.3\%$ hit rate over its LRU on the application stream at a matched 24k-token budget (Qwen2.5-0.5B, 8 concurrent requests), at neutral latency, by the engine's own counters. SGLang on the same stream (Qwen3-8B-FP8, 12 concurrent requests) has an LRU baseline within half a point (0.577 vs. 0.572) and reaches $+9.2\%$. Branchiness is inert in both engines, so the difference reflects engine-level factors, such as vLLM's 16-token blocks, and the different model and concurrency.

F. Mechanism, workload dependence, and the advisor

Why persistence: an analysis. Under the independent reference model (block i referenced with fixed probability p_i), the optimal demand policy for uniform-size items evicts the resident item with the lowest p_i [14], and persistent counts converge to the ranking of p_i, so persistent LFU converges to that policy. The prefix constraint does not conflict with it: every reference to a block also references its ancestors, so persistent counts satisfy $f(\text{parent}) \geq f(\text{child})$, and for a capacity of C blocks the C most frequent blocks (ties broken toward ancestors) form a prefix-closed set that leaf-only eviction can maintain. A per-node count instead restarts at each admission: a block whose reuse distance exceeds its residence time is always readmitted with a count of one and ranks with one-shot blocks, so in-cache LFU degrades toward LRU, whereas for reuses within the residence time the two counts coincide. This predicts both observed regimes: in-cache LFU near LRU on the traces, whose reuse distances reach thousands of requests (p90 above 1,600), and stock LFU matching PREFER live, where p90 is below 90 requests (Table I). Real traces are not independent references, which is why the conversation trace keeps a moderate recency weight.

One-shot pollution. A prefix-closed cache fed a mix of reused prefixes and never-again content is polluted by the latter: LRU admits every unique block and, because it was just touched, keeps it long enough to evict a reused prefix. Persistent frequency starves one-shot blocks and retains reused ones. An admission-control probe confirms the mechanism: caching a block only on its *second* reference nearly doubles LRU on conversation ($+90\%$ at capacity 2048), matching the searched policy, and stacking it on frequency adds nothing.

When LRU wins. In the burst loop an active session steps again within a few requests and finished sessions never return,

so recency predicts reuse and accumulated counts can protect contexts that are no longer needed; at matched budgets the same agents lose under PREFER without cool-downs but win with them. No single statistic in Table I separates the regimes: tool/agent has 58% near reuses yet favors frequency, and the multi-tenant templates recur within a median of 5 requests yet are evicted by LRU under the unique-context stream.

Advisor. We therefore select by simulation: given a workload sample, the advisor simulates LRU and PREFER at capacities scaled to the sample's working set and recommends PREFER only if the best simulated gain is at least 5%, else LRU. The 5% margin was fixed when the advisor was written, before its validation, as a practical-significance floor below which a non-default policy is not worth deploying; it was not tuned. Validated against the live outcomes (multi-tenant, resumption, burst loop, and the skewed application stream), it picks the live winner in all four, including recommending *against* PREFER on the burst loop (predicted -9.9%, live -15.3%/-7.4% at 6k/8k). The four decisions are identical for any threshold between 0 and 12.7%; the application's one-pass evaluation (predicted $+5.2\%$) is flagged as marginal. Four families is a small validation set, and the advisor's predicted magnitudes overestimate live gains; we claim correct decisions, not calibrated magnitudes.

VII. Negative Results and Generalization

Learning does not help. A gradient-boosted reuse-distance predictor (held-out Spearman rank correlation 0.38–0.80), used as an eviction scorer, loses to the linear policy in 13 of 14 trace–capacity settings: optimizing next-use accuracy is not optimizing hit rate. **Non-linearity does not help.** Adding seven interaction terms and searching directly for hit rate ties the linear policy (0.633 vs. 0.636 test fraction-of-reference). **Generalization.** On a synthetic Zipf generator, frequency beats LRU at every popularity skew (+110% at Zipf a=0.4, +9% at a=1.6); on a workload built from public multi-turn conversations (28.8% reuse), it beats LRU at every capacity (+100% at 132 blocks, +7.8% at 552).

VIII. Limitations

The oracle-style numbers are relative to a feasible reference, not the optimum. All live runs use one consumer GPU, mostly at concurrency 1 (8–12 for the fidelity, cross-engine, and 8B application runs; 1–64 in the load sweep); the large simulated gaps need production-scale working sets that a 16 GB GPU cannot host, so live margins are measured on constructed workloads, not on replays of the production traces; at their short reuse distances the live margin of persistent over per-node counts is not significant, so the large persistent-versus-in-cache gap rests on simulation; a live test at production reuse distances needs a larger GPU. The load sweep, cross-engine, and application results are single runs per configuration, and the application results use text-only proxy models with placeholder tokens for the images. The traces come from one provider and cover about an hour each, so how far their reuse structure represents other services is untested, and the advisor is validated on only four workload families. LIRS matches PREFER at mid-range capacity, so the advantage over adaptive policies is regime-specific. Multi-node and disaggregated serving remain open.

IX. Conclusion

LRU leaves much of the achievable prefix-cache hit rate unused in the cache-constrained regime. An automated search converges on frequency-dominated eviction, and comparison with LFU, ARC, and LIRS shows that the operative ingredient is frequency that persists across evictions, which shipping engines discard. Live, frequency eviction gains 25–31% hit rate over 8 seeds on two frequency-favorable workloads; at their short reuse distances the engine's built-in LFU already captures it, so enabling it is a practical first step. Persistent counts, which the long-reuse traces require, add no significant gain there. Frequency eviction carries to a second engine and a VLM application, and converts to up to 10.5% more throughput on an 8B model under load. Where recency wins, an advisor that simulates the workload keeps LRU.

References

[1] W. Kwon *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. ACM SOSP*, 2023.

[2] L. Zheng *et al.*, "SGLang: Efficient execution of structured language model programs," in *Proc. NeurIPS*, 2024.

[3] I. Gim *et al.*, "Prompt cache: Modular attention reuse for low-latency inference," in *Proc. MLSys*, 2024.

[4] B. Gao *et al.*, "Cost-efficient large language model serving for multi-turn conversations with CachedAttention," in *Proc. USENIX ATC*, 2024.

[5] P. Cao and S. Irani, "Cost-aware WWW proxy caching algorithms," in *Proc. USENIX Symp. Internet Technologies and Systems*, 1997.

[6] N. Megiddo and D. S. Modha, "ARC: A self-tuning, low overhead replacement cache," in *Proc. USENIX FAST*, 2003.

[7] S. Jiang and X. Zhang, "LIRS: An efficient low inter-reference recency set replacement policy to improve buffer cache performance," in *Proc. ACM SIGMETRICS*, 2002.

[8] L. Breslau, P. Cao, L. Fan, G. Phillips, and S. Shenker, "Web caching and Zipf-like distributions: Evidence and implications," in *Proc. IEEE INFOCOM*, 1999.

[9] G. Einziger, R. Friedman, and B. Manes, "TinyLFU: A highly efficient cache admission policy," *ACM Trans. Storage*, vol. 13, no. 4, 2017.

[10] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, 1966.

[11] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. ACM/IEEE ISCA*, 2016.

[12] Z. Song, D. S. Berger, K. Li, and W. Lloyd, "Learning relaxed Belady for content distribution network caching," in *Proc. USENIX NSDI*, 2020.

[13] R. Qin *et al.*, "Mooncake: Trading more storage for less computation—A KVCache-centric architecture for serving LLM chatbot," in *Proc. USENIX FAST*, 2025.

[14] E. G. Coffman, Jr. and P. J. Denning, *Operating Systems Theory*. Englewood Cliffs, NJ: Prentice-Hall, 1973.